TraceBack — Failure and Recovery Test Report

Nov 6, 2026 · @Soumyaranjan

Result: in all 10 failure scenarios TraceBack failed safely and recovered on its own. No confirmed data was lost, no request hung, and every part came back without manual repair, in under 2 seconds after the failed part returned. The first run found 4 recovery defects; all 4 were fixed, and the run below is on the final code.

TraceBack is the IIT Kanpur Lost & Found portal: a Next.js website, a FastAPI backend (two processes), a background worker and PostgreSQL, behind Caddy on one AWS server. These tests take parts away while it runs: the database, the API, the worker, the AI providers, the mail server, and finally everything at once.

1. Approach

- **Where:** a copy of the full system on a fresh test database, set up as in production: two API processes, the AI queue only in the worker, API docs off. The live site was not touched.
- **How:** `testing/failure_recovery_tests.py` starts the app, signs in a student, a finder, an owner and a guard, then injects each failure while requests are running. It stops PostgreSQL cleanly and like a power cut, kills processes with `kill -9`, points the AI providers at a closed port, and replaces Gmail with a local SMTP server (`fake_smtp.py`) that it can switch off.
- **What is checked:** how fast an error comes back and what it says; whether anything is lost or half-saved (by reading the database afterwards); and how long until TraceBack works again, without restarting anything except the part that failed.
- **Pass rule:** errors within 3 s (10 s for email, which retries), no confirmed write lost, no half-saved record, and full recovery without manual repair.

2. Results

ID	Failure injected	Expected	Observed	Recovery	Result
FR-01	PostgreSQL stopped while a student is signed in	Requests fail fast with a clear message; nothing half-saved; the API reconnects by itself	Health check, Found page and a new report all answered 503 “TraceBack is briefly unavailable” in 0.01–0.02 s; no internals shown; 0 half-saved reports; the session still worked afterwards	Healthy as soon as PostgreSQL was up; API not restarted	Pass
FR-02	PostgreSQL stopped for 3 s while 10 students browse	Fast failures during the outage; every request after it succeeds	11,228 requests in 30 s; 2,411 failed during the outage (2,407 with a clear 503, 4 connections cut mid-query); the last failure started 0.1 s before the database was back; slowest request 0.21 s	4,406 requests after the restart, 0 failed	Pass
FR-03	Both API processes killed (`kill -9`)	Website answers at once instead of hanging; nothing lost	Website answered in 0.03 s while the API was down; after the restart the session, the item and its photo were all served	API healthy 0.8 s after starting (Docker restarts it on the server)	Pass
FR-04	API killed while 8 students send reports	Every confirmed report saved; nothing half-saved	333 reports confirmed, 8 requests cut off; 336 saved; 0 confirmed reports missing; 0 reports without their AI job	Restarted; all 336 complete	Pass
FR-05	Worker stopped while 10 reports come in	Reports still accepted; AI work waits and is done once	10 reports accepted and visible; 10 AI jobs waited in the queue; all 5 lost reports got their match; 10 jobs in all	Queue empty 7.3 s after the worker started	Pass
FR-06	Worker killed while it is working on an AI job	The interrupted job runs again	The job was taken again (try 2) and finished; the report was analysed	0.9 s after the worker restarted	Pass

ID	Failure injected	Expected	Observed	Recovery	Result
FR-07	Groq and Gemini both unreachable	Matching falls back to rules; the failure is logged	Job done in 3.1 s; agent run logged as “fallback” with both errors; rule-based match saved; the owner’s panel showed 5 suggestions	No recovery needed	Pass
FR-08	No mail server, then back, then down for 1.5 s mid-send	Clear “try again”; immediate retry possible; short outages bridged	503 “We couldn’t send the email just now” in 3.05 s, logged as failed; retry at once: 202, code delivered and accepted; 1.5 s outage: delivered by the automatic retry (202 in 3.08 s)	Immediate	Pass
FR-09	No mail server when a guard approves a claim	The approval is saved; the failed email is logged	Approval 200 in 3.04 s; the student sees APPROVED; the email is logged as failed for staff	Immediate	Pass
FR-10	Power cut: API, worker, website and PostgreSQL killed together	Everything starts again; no data lost; users stay signed in	Rows before and after identical (users, items, claims, photos); session and photo served; schema at the latest migration; worker running	Back up in 1.7 s	Pass

Raw results: `testing/results/failure_recovery_results.json` and `failure_recovery_run.log`.

3. Defects found and fixed

The first run passed 6 of the 10 scenarios. The 4 failures were real recovery defects; each was fixed with a test that reproduces it (`backend/tests/test_failure_recovery.py`, 10 tests), and the full run then passed 10 of 10.

Defect	Found by	Severity	Fix
D-19: with the database down, requests got a bare “Internal Server Error” (500) and some connections were dropped	FR-01	Medium	A handler answers 503 with a clear message and `Retry-After`; the pool reconnects when the database is back
D-20: a report was committed before its AI job, so a crash in between left a report that was never analysed (1 of 313 reports)	FR-04	Medium	Report and job are saved in one commit; a failing matcher still never loses the report (savepoint)
D-21: an AI job interrupted by a killed worker was marked failed for good, so the report never got matches and its page kept saying “looking for matches”	FR-06	High	Interrupted jobs are queued again, up to 3 tries, judged by a new `started_at` time; after 3 tries the page says matching failed
D-22: a sign-in code that could not be emailed still counted for the 60-second wait, so “try again” was answered “please wait” (429)	FR-08	Medium	The unsent code is removed, so the student can try again at once

4. What makes TraceBack recover

- **Restart policy:** every container runs with `restart: unless-stopped`, so Docker restarts a crashed API, worker, website or database; the API waits for the database health check before it starts.
- **Database connections:** the pool checks each connection before use (`pool_pre_ping`), so after a database restart the API reconnects without being restarted. While the database is down, requests get a 503 at once.
- **Atomic writes:** each action is one transaction; a report and its AI job are saved together; a crash leaves either all of it or none.
- **Durable queue:** AI work waits in the `ai_jobs` table, taken with `FOR UPDATE SKIP LOCKED`; interrupted jobs run again, up to 3 tries.
- **Outside services are optional:** if Groq and Gemini fail, matching falls back to rules; email retries twice on a temporary failure, logs every attempt, and never undoes a guard’s decision.
- **Sessions in the database:** users stay signed in through restarts and power cuts.

5. Limits

- Tested on a copy with 2 CPUs, not on the AWS server; on the server, Docker's restart adds a few seconds that a manual restart in the test did not. A reboot test on the live server is part of the deployment checklist (Deployment tab).
- Disk-full and network partitions between containers were not simulated.
- PostgreSQL 16 in the test, 17 in production.

6. How to re-run

```
pip install aiosmtpd
PG_STOP="sudo systemctl stop postgresql" PG_START="sudo systemctl start
postgresql" \
PG_KILL="sudo pkill -9 -f postgres" python testing/failure_recovery_tests.py
```

The script creates and drops its own database (`lostfound_test_failure`) and takes about 3 minutes. Never point it at real data.